Towards Benchmarking Agentic Data Scientists

Kartik Nagaraj Nayak[1,3,*], Nitishkumar Solpure[1,*], Niladri Mitra[1,*], Tom Zehle[1,2], Alexander Pfefferle[1,2], Omar Swelam[1,2], Frank Hutter[4,2,1]
[1]University of Freiburg [2]ELLIS Institute Tübingen [3]Zuse School ELIZA [4]Prior Labs
[*]Equal contribution.

Abstract

Agentic machine learning systems leverage large language models to autonomously design, execute, and refine machine learning pipelines. Despite rapid progress in this area, systematic evaluation of these systems remains limited. In this work, we benchmark four representative approaches: MLZero, AgentK, AIDE, and an open-source reimplementation of MLE-STAR. Additionally, we use AutoGluon as an AutoML baseline. We use a unified evaluation framework with fixed data splits and standardized metrics. Beyond predictive performance, we evaluate execution reliability, robustness, and failure behavior. Our results show that ensemble-based systems such as AutoGluon provide the most stable and consistent performance across heterogeneous tasks, while agentic approaches occasionally achieve strong task-specific results but exhibit higher variance and frequent execution failures. The study highlights important trade-offs between reasoning-based automation, robustness, and practical usability, and provides a reproducible benchmark for future research on agentic machine learning systems. Codebase available at `github.com/KARTIK-N-NAYAK/Benchmarking-Agentic-Data-Scientists`.

1 Introduction

Agentic machine learning systems mark a shift from traditional static AutoML pipelines toward adaptive, decision-making workflows. Rather than producing a single model directly, these systems set goals, plan actions, execute code, observe outcomes, and iteratively refine their behavior, enabling a more flexible and autonomous approach to machine learning.

Benchmarking such systems is important because different agents may solve the same task in different ways, leading to large differences in predictive performance, robustness, runtime, and failure modes. In addition, results can depend strongly on the backbone LLM, prompting strategy, and execution environment, making standardized evaluation essential for understanding what these systems can reliably achieve.

In this work, we evaluate four agentic systems: AgentK [Grosnit *et al.*, 2025], AutoGluon Assistant, also known as MLZero [Fang *et al.*, 2025], AIDE [Jiang *et al.*, 2025], and MLE-STAR [Nam *et al.*, 2025], for which we use an open-source reimplementation [WalkingDevFlag, 2025]. Alongside them we evaluate AutoGluon [Erickson *et al.*, 2020], a mature and widely deployed AutoML framework that consistently ranks among the strongest methods on tabular benchmarks. It serves as the bar an agentic system must clear: if reasoning over a task does not beat a well engineered fixed search, the additional machinery is not yet earning its cost. Descriptions of each system are given in Section 3.2.

Evaluating these systems presents practical challenges, including heterogeneous dependencies, different backbone LLMs, and substantial computational constraints, all of which can affect reproducibility and fairness. To address this, we propose a unified benchmarking framework that standardizes datasets, execution, and evaluation metrics across all systems. Using a curated suite of tabular classification and regression datasets, we compare not only predictive performance but also robustness under a controlled protocol.

We argue that current agentic ML systems do not yet live up to their promise. While reasoning-based pipeline generation can achieve strong task-specific results, none of the evaluated systems approaches the reliability and consistency of a mature AutoML baseline. Our experiments show that they are highly sensitive to design choices and execution settings. AIDE benefits from previous run logs and often improves across iterations, indicating strong adaptive behavior. AgentK produces workable pipelines on the smaller tasks, but both its predictive performance and its execution reliability degrade on larger datasets, where it accounts for a substantial share of the failures we observe. Our benchmark provides a reproducible basis for tracking when that gap closes.

Our contributions are three. We release a unified execution and evaluation framework that runs five systems under one protocol on fixed splits. We audit every fold for target leakage and report what removing the two affected datasets changes. And we define a robustness-adjusted score that combines predictive performance with cross-fold stability and execution reliability, and we measure what the reliability term contributes to the ranking.

Agentic ML systems are moving targets: their prompts, decoding settings, and tool scaffolds are co-designed around specific backbone LLMs, while new state-of-the-art LLMs and tabular foundation models appear every few months. Keeping

such systems well configured by hand is not sustainable, and their configuration should itself be treated as an automated optimization problem.

2 Related Work

Traditional AutoML frameworks such as AutoGluon, Auto-sklearn [Feurer *et al.*, 2020], and TPOT [Olson and Moore, 2016] automate model selection, feature engineering, hyperparameter tuning, and ensembling within predefined search spaces. These systems achieve strong performance on structured data, but they typically rely on static optimization loops and offer limited capability for adaptive reasoning. Other automated optimization paradigms, including neural architecture search (NAS) [Zoph and Le, 2017] and hyperparameter optimization (HPO) [Bergstra *et al.*, 2011], further improve automation, yet they still operate within rigid search procedures.

The emergence of large language models has enabled a shift toward *agentic ML*, where systems iteratively write, execute, debug, and refine code for machine learning workflows. Early work such as MLAgentBench [Huang *et al.*, 2024] demonstrated the value of prompting strategies like ReAct [Yao *et al.*, 2023] and Reflexion [Shinn *et al.*, 2023] for guiding agents through data exploration and model development. More recent systems extend these ideas into more autonomous frameworks for end-to-end ML engineering, highlighting a new trade-off between flexibility, specialization, and robustness.

Benchmarking has long been central to machine learning research, with platforms such as OpenML [Vanschoren *et al.*, 2014] and GLUE [Wang *et al.*, 2019] providing standardized evaluation protocols. In tabular ML, benchmarks have traditionally compared model families and AutoML systems under controlled settings, but agentic ML introduces new evaluation challenges because success depends not only on predictive quality but also on robustness, runtime, and execution reliability. Recent efforts such as MLE-bench [Chan *et al.*, 2025] and MLGym [Nathani *et al.*, 2025] assess agent performance at a higher level, often using task success rates or leaderboard scores. However, these metrics do not fully capture end-to-end ML pipeline quality, and differences in backbone LLM, prompt sensitivity, and stochastic inference make reproducible comparison difficult.

In contrast to prior work, our study provides a unified benchmarking framework tailored to agentic ML systems on tabular data. By standardizing datasets, execution conditions, and evaluation metrics across multiple agents and backbone LLMs, we enable a controlled comparison of predictive performance, reliability, and practical usability. This framework fills a gap between traditional AutoML benchmarking and emerging LLM-agent evaluation, and it provides a reproducible basis for future research in agentic machine learning.

3 Methodology

We present a unified benchmarking framework for evaluating agentic machine learning systems on tabular prediction tasks. The framework is designed to support a fair, controlled, and reproducible comparison across systems with different internal architectures and execution strategies. It consists of a standardized execution pipeline, a curated dataset suite, and a common evaluation protocol.

3.1 Pipeline Setup

Our benchmarking pipeline is built around three modular components: dataset handlers, agent interfaces, and a central execution engine. Dataset handlers provide a consistent view of each task instance, while agent interfaces expose a standardized entry point for system-specific execution. The execution engine coordinates the interaction between the agent and the dataset, launches runs under identical conditions, and records the resulting predictions and failure modes.

Concretely, for each run the execution engine materializes the fold's train and test files at a fixed location, launches the agent inside the prepared environment with the task description for that dataset, and waits for it to terminate within its budget. On termination it collects the agent's prediction file, checks that the file parses and provides a prediction for every test row, and records the run's exit status. A run is valid if and only if that check passes. Every other outcome is counted as a failure.

This design removes the need for custom evaluation logic for each system and ensures that all agents receive the same input artifacts and are assessed using the same downstream scoring procedure. The framework is implemented in a controlled environment with fixed software dependencies to reduce variability due to external factors. Its modular structure also makes it straightforward to extend the benchmark with additional agents or datasets in future studies.

3.2 Agents

We evaluate four agentic systems and one non-agentic AutoML baseline, covering a range of design choices in search, reasoning, and workflow construction.

MLZero [Fang *et al.*, 2025] uses Monte Carlo Tree Search (MCTS) to explore the space of machine learning pipelines. This search-based strategy balances exploration and exploitation, enabling structured pipeline generation.

AgentK [Grosnit *et al.*, 2025] is an LLM-driven agent that constructs machine learning workflows through iterative reasoning and tool use. It generates candidate solutions, evaluates intermediate outputs, and revises its strategy based on feedback.

AIDE [Jiang *et al.*, 2025] treats machine learning engineering as an iterative code-refinement process. It uses evaluation feedback and execution logs to progressively improve generated solutions over multiple rounds.

MLE-STAR [Nam *et al.*, 2025] follows a structured agentic workflow that combines planning, execution, and evaluation. Its design emphasizes modular reasoning and targeted refinement of the machine learning pipeline. We use an open-source reimplementation [WalkingDevFlag, 2025]. Appendix A records how it differs from the original release.

AutoGluon [Erickson *et al.*, 2020] serves as a strong non-agentic baseline. It is a widely used AutoML framework based on ensemble learning and heuristic-driven optimization, and it provides a useful reference point for comparing agent-based systems against a mature automated approach.

3.3 Datasets

We construct the benchmark from a collection of 23 open-source tabular datasets sourced primarily from Kaggle and other publicly available repositories. To improve relevance and reliability, we manually curate the dataset suite by inspecting dataset quality and excluding benchmarks with excessive missing values, video data, synthetic data, and other data integrity issues. Applying these filtering criteria reduced the benchmark to 12 datasets. A subsequent target-leakage audit of every fold removed a further two datasets, giving a final evaluation suite of 10.

The selected datasets span both regression and classification tasks and exhibit substantial diversity in dataset size, feature types, class balance, and overall difficulty. This variation enables evaluation not only of predictive performance but also of robustness across different problem settings. Each dataset is partitioned into three fixed, non-overlapping folds, and every fold is further divided into train and test splits. We use three folds per dataset to keep total compute tractable given per-run budgets of up to 24 h across five systems. Section 6 discusses the precision of the resulting variance estimates. The same predefined splits are reused across all systems to minimize variance caused by data partitioning and ensure fair and reproducible comparison across methods. The evaluation suite is listed in Appendix C.

3.4 Leakage Audit

We audited all 36 folds of the original 12 datasets for target leakage before finalizing this version, checking for the target column itself in the test file, exact reconstruction of the target from any single feature or additive pair of features, schema mismatches between the train and test files of a fold, and row overlap between the splits. Two datasets failed and were removed. In *Bike Sharing Demand* the features `casual` and `registered` are retained in the test file and sum exactly to the target `count`, reproducing the held-out target on every test row. In *Bioresponse* the test file of the first fold retains the `Target` column itself. The consequence is not a uniform loss of signal: on *Bike Sharing Demand* the two systems that exploited the leak, AIDE and MLZero, ranked above the two that did not, AutoGluon and AgentK, so the dataset was ordering systems by their propensity to discover leakage and was penalizing the non-agentic baseline. Benchmarks for agentic systems are particularly exposed to this failure, because an agent that reads the data is far better placed to find an artifact than a fixed search space is, so leakage does not degrade all systems equally and cannot be assumed to cancel out. Per-fold forensics are given in Appendix B.

3.5 Evaluation Protocol

All systems are evaluated under the same protocol using identical dataset splits and scoring procedures. Results are aggregated across the three splits for each dataset, providing a more stable estimate of performance than a single train-test partition.

We use Root Mean Squared Error (RMSE) and coefficient of determination (R^2) for regression tasks, and ROC-AUC for classification tasks. These metrics capture both predictive performance and ranking quality, depending on the task formulation. No task-specific tuning is performed for individual systems, so the benchmark reflects each method's default behavior under a common evaluation setting. All experiments involving agentic workflow generation and reasoning were performed using `gpt-oss-120b` [OpenAI, 2025] as the backbone LLM. The same model configuration was used across all evaluated agents to ensure a fair comparison.

To ensure consistency across all evaluated systems, each agent received standardized descriptions containing the task instructions, evaluation metric descriptions, and dataset-specific descriptions. These descriptions were identical across methods and were used as the primary task specification during execution. They are part of the released benchmark code at `github.com/KARTIK-N-NAYAK/Benchmarking-Agentic-Data-Scientists`.

3.6 Execution Environment and Budgets

Hardware. Every run was executed as a single scheduled job on a shared GPU cluster, allocated 8 CPU cores and one NVIDIA RTX 2080 Ti. The allocation was identical for all systems, including those that make no use of a GPU.

Language model. All agentic systems used `gpt-oss-120b`, served locally within our institution behind an OpenAI-compatible endpoint. Decoding parameters were each system's repository defaults rather than a common setting. For AgentK, whose model configuration is committed in the benchmark repository, decoding was greedy with a 2048-token generation limit and a 16,384-token context. We chose defaults over a forced common setting because these systems co-design their prompting and decoding, and overriding one without the other would measure our reconfiguration rather than the system. The cost is that decoding is not held constant across systems, which we record as a limitation.

Network access. Compute nodes could reach the internal model endpoint and nothing else. Agents could not install packages or retrieve external resources at run time. Environments were fixed in advance with pinned dependencies.

Time budgets. Budgets were not uniform, and the differences were deliberate. AutoGluon was given 7200 s per run. As a fixed-search AutoML system rather than a code-generating agent, it has no use for a longer budget, and in fact converged before exhausting it on two of the ten datasets. Its `best_quality` preset enables dynamic stacking, which reserves a quarter of the budget for a holdout sub-fit, leaving roughly 5400 s for the final fit. AgentK was capped at 14,400 s, with an internal budget of 18,000 s total and 600 s per candidate submission. The cap was imposed because AgentK does not terminate once it stops making progress and would otherwise hold an allocation idle. The remaining agentic systems were capped at 24 h.

Retries and the definition of failure. The execution engine performed no automatic retries. Each agent's own error-recovery loop ran as designed. When a run terminated without producing output, it was relaunched manually. Relaunching recovered failures attributable to the execution environment but never recovered a run that failed within the agent itself. A run

is counted as failed when it produced no parseable prediction file for its fold on any attempt.

4 Results

The experimental evaluation demonstrates substantial differences in the robustness, consistency, and predictive performance of the evaluated automated machine learning agents across diverse tabular classification and regression benchmarks. Overall, AutoGluon achieved the best average rank, followed by AIDE, MLE-STAR, AgentK, and MLZero as shown in Table 1. Average ranks were computed using a robustness-adjusted scoring scheme that combines predictive performance, cross-fold stability, and execution reliability.

#	Method	Average Rank ↓
1	AutoGluon	$\mathbf{1.75 \pm 1.36}$
2	AIDE	2.75 ± 1.23
3	MLE-STAR	3.15 ± 1.06
4	AgentK	3.55 ± 1.54
5	MLZero	3.80 ± 1.14

Table 1: Overall average rankings of the evaluated automated machine learning systems across the evaluation suite. Rankings were computed using robustness-adjusted scores derived from predictive performance, cross-fold stability, and execution reliability. Lower ranks indicate better overall performance. Values are reported as mean rank ± standard deviation across datasets.

4.1 Robustness-Adjusted Score

Throughout, robustness refers jointly to the stability of performance across folds and to execution reliability, the ability to produce a valid result at all.

The scoring function used initially, and reported in the submitted version of this work, combined the mean μ of a metric across folds, its standard deviation σ, and the reliability r as $\mu \cdot r/(1+\sigma)$. We have dropped it, because it is defective in three independent respects. First, its *sign* is wrong when performance is negative: R^2 may fall below zero, and for a negative μ the division by $1+\sigma$ makes the score increase with variability, so an unstable system is rewarded for being unstable. Second, its *direction* is wrong for error metrics: RMSE is better when smaller, but the score is monotonically increasing in μ, so on the regression datasets it ranks the worst system first. Third, its *scale* is wrong: σ carries the units of the metric, so the penalty term $1+\sigma$ is negligible for ROC-AUC, where $\sigma \ll 1$, and overwhelming for an RMSE in the millions. The three quantities are consequently not comparable across datasets, and the aggregate ranking is dominated by whichever datasets happen to have the largest-magnitude targets.

The formulation below replaces it. Each of the three factors is rescaled to be dimensionless, bounded and correctly signed, which removes one defect apiece. All results reported in this paper are computed under the corrected score.

For each dataset, we combine the three quantities multiplicatively as

$$\text{score} = \frac{p \cdot r}{1+s},$$

where p is normalized performance, s is normalized spread across folds, and r is reliability, the fraction of folds that produced a valid result. Each factor is dimensionless and bounded, so scores are comparable across datasets and across metrics. A system that completes only a single fold has $s = 0$ by construction, but its reliability factor $r = 1/3$ then dominates, so an isolated lucky success is penalized rather than rewarded.

Normalization follows the AMLB convention [Gijsbers *et al.*, 2024] of rescaling a metric between a trivial baseline and its ideal value, using fixed anchors rather than the observed range. ROC-AUC is anchored at random ranking and perfect ranking, $p = \text{clip}((\mu - 0.5)/0.5,\ 0,\ 1)$ with $s = \sigma/0.5$. R^2 is anchored at the mean predictor and perfect prediction, $p = \text{clip}(\mu,\ 0,\ 1)$ with $s = \sigma$. RMSE has no such natural anchor and is unbounded, so it is expressed relative to the best result obtained on that dataset, $p = \min_j(\mu_j)/\mu$, with the coefficient of variation $s = \sigma/\mu$ as its spread. This makes the direction of each metric explicit: for RMSE, lower is better, and the ratio is formed so that a better result yields a larger p. Dataset-wise ranks are computed from the corrected scores and averaged across the evaluation suite.

Across classification tasks, AutoGluon and AIDE consistently achieved the strongest performance. Among the systems that completed every fold, AutoGluon obtained the highest ROC-AUC on all but one classification dataset, demonstrating strong generalization capability on structured tabular data. AIDE achieved highly competitive results. MLZero achieved the highest scores on several datasets. However, these results frequently corresponded to runs with zero reported variance caused by only a single successful execution. Consequently, although MLZero occasionally achieved strong isolated performance, its overall robustness and reliability were substantially weaker than AutoGluon and AIDE. MLE-STAR delivers moderate performance across the benchmark, but generally trails the best-performing methods, while AgentK consistently underperformed relative to the stronger baselines, exhibiting lower predictive performance and multiple execution failures. In regression tasks, performance varied significantly depending on dataset characteristics. AutoGluon achieved the best overall performance. No system attains a near-perfect result on any regression dataset in the audited suite: the highest R^2 recorded is 0.862. The near-perfect regression results reported in the submitted version were obtained exclusively on *Bike Sharing Demand* and were a consequence of the leak described in Section 3.4, which is a further reason to treat that dataset as uninformative rather than merely noisy. However, MLZero also exhibited extreme instability on several datasets, where RMSE variance increased dramatically. MLE-STAR demonstrated competitive performance on several regression datasets. AgentK showed inconsistent performance, often producing unstable predictions with large variability across folds and demonstrating weaker generalization compared to the stronger ensemble-based baselines.

Failure analysis further highlighted substantial differences in robustness as described in Table 2. AutoGluon and MLE-STAR achieved a 100% success rate across all benchmark runs without any execution failures. In contrast, AgentK and MLZero experienced high failure counts. Although AIDE achieved strong predictive performance, it also exhibited sev-

Method	Failures	Success Rate (%)
AutoGluon	0	**100.0**
AgentK	11	63.33
AIDE	5	83.33
MLZero	15	50.00
MLE-STAR	0	**100.0**

Table 2: Failure statistics across all 30 benchmark runs. Failure counts denote the number of runs that did not produce valid outputs, on any attempt.

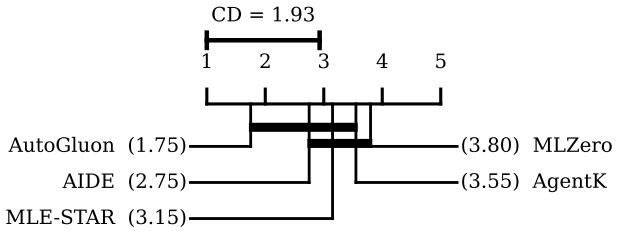


Figure 1: Critical Difference plot of average robustness-adjusted ranks across the audited evaluation suite. Lower is better. Horizontal bars connect methods whose ranks are not significantly different under the Nemenyi post-hoc test (CD = 1.93).

eral execution failures. These results indicate that predictive performance alone is insufficient for evaluating autonomous machine learning agents, and robustness, stability, and execution reliability are equally important considerations. Overall, the results suggest that ensemble-based automated machine learning systems with strong execution reliability and stable generalization, such as AutoGluon, currently provide the most consistent performance across heterogeneous tabular learning tasks.

5 Analysis and Discussion

We further evaluated statistical significance using the Friedman test and the Nemenyi post-hoc test, together with exploratory pairwise Wilcoxon signed-rank tests on per-dataset method ranks. The Friedman test indicated statistically significant differences among methods ($\chi^2 = 10.46$, $p = 0.033$, over 10 datasets and 5 methods), confirming that the observed ranking differences are unlikely to be due to random variation alone.

Pairwise Wilcoxon signed-rank tests on per-dataset ranks are exploratory only: three comparisons reach nominal significance at $\alpha = 0.05$, but none survives correction for multiple comparisons under either Holm or Benjamini and Hochberg, so we make no pairwise claims.[1] The difference from the submitted version, which reported $p = 0.007$ on 12 datasets, is explained by the corrected score and the removal of the two leaked datasets: at a fixed 12 datasets the corrected score moves p to 0.054, and dropping the leaked datasets brings it to the 0.033 reported here.[2] Evidence for differences between methods therefore rests on the Friedman and Nemenyi tests, which account for multiplicity by construction, and on the effect sizes visible in the average ranks.

Figure 1 presents the Critical Difference (CD) diagram computed from the robustness-adjusted average ranks across all evaluated datasets. The ranking procedure combines predictive performance, cross-fold stability, and execution reliability into a single robustness-aware score. Lower ranks correspond to stronger overall performance. The plot shows that AutoGluon and AIDE achieve the strongest overall rankings and are not statistically significantly different from each other. MLE-STAR achieves intermediate performance, remaining competitive with several agentic approaches despite trailing the strongest ensemble-based systems. In contrast, AgentK and MLZero generally obtain weaker average rankings due to lower robustness, higher variability across folds, and increased failure frequency. Horizontal bars indicate groups of methods whose differences are not statistically significant under the Nemenyi post-hoc test.

The performance differences between agents primarily stem from differences in search strategy, model selection, and execution robustness. Ensemble-based systems such as AutoGluon generally achieved the most stable and consistent performance across datasets due to their ability to combine diverse models and reduce variance. In contrast, methods such as AIDE and MLZero occasionally achieved stronger task-specific performance by rapidly converging to effective pipelines through iterative refinement or targeted search, but their performance was less consistent across heterogeneous datasets.

5.1 What Reliability Contributes to the Ranking

The premise of this benchmark is that predictive performance alone is an insufficient basis for comparing autonomous systems. That premise can be measured rather than asserted, by re-ranking the same results on mean performance alone and observing what moves.

Figure 2 shows the result. Dropping the stability and reliability terms moves MLZero from fifth place to second, a gain of more than a full rank position, while MLE-STAR falls from third to last. The mechanism is visible in the per-dataset results: MLZero's strongest figures come from datasets on which only one of its three folds completed, so its standard deviation is zero by construction and its mean is computed over a single run that happened to succeed. A ranking that reads only the mean rewards precisely this, and cannot distinguish it from a system that succeeded reliably at a similar level.

We report both rankings rather than only the one that supports our argument. The ordering of the top system is unaffected, since AutoGluon ranks first under either scheme, but the ordering of the agentic systems among themselves is not stable to this choice, and a reader who cares only about peak

[1] The three nominally significant comparisons are AutoGluon against MLZero, MLE-STAR and AgentK, with raw p-values of 0.018, 0.039 and 0.041. Under Holm the smallest adjusted value is 0.176, and under Benjamini and Hochberg it is 0.137, so zero of ten comparisons remain significant. Full values are given in Table 4.

[2] The earlier value was smaller because the uncorrected score collapsed to approximately μ/σ on the error-metric datasets, and inverse variability is far more consistent across datasets than performance is, which is precisely what the Friedman statistic rewards.

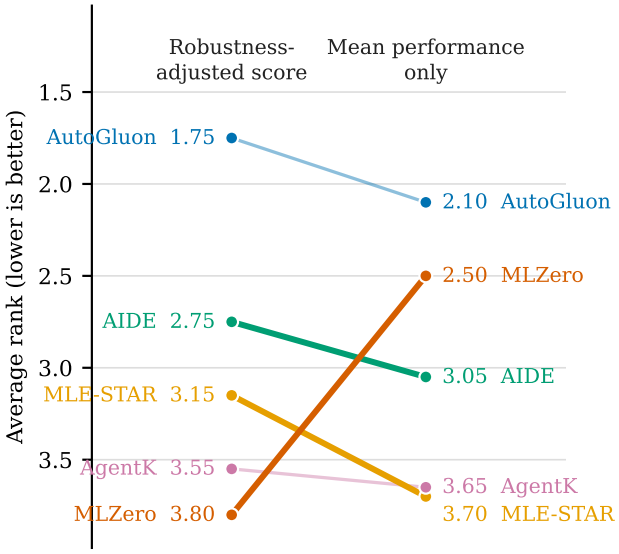


Figure 2: Average ranks under the full robustness-adjusted score (left) and under mean predictive performance alone (right), on the audited evaluation suite. Lower is better. MLZero rises from fifth to second when stability and reliability are removed from the score, because several of its strongest results come from datasets on which only one fold completed and its cross-fold standard deviation is therefore zero.

predictive performance would reasonably reach a different conclusion about MLZero than a reader who cares about obtaining a result on every attempt. Making that dependence explicit is more useful than concealing it behind a single number.

A common pattern observed across experiments is that ensemble-based approaches generally achieved stronger and more stable performance on larger datasets with sufficient training data, where combining diverse learners improved robustness and predictive performance. In particular, boosting-based and ensemble methods performed well on structured tabular datasets containing complex nonlinear feature interactions and heterogeneous feature distributions. For example, *AutoGluon* consistently achieved strong performance and low variance on datasets such as *Give Me Some Credit* and *Sberbank Russian Housing Market*, where ensemble diversity improved generalization across folds. In contrast, methods relying on a single high-capacity model or aggressive optimization often exhibited overfitting, higher variance, and reduced generalization on smaller or noisier datasets. This behavior was particularly visible for *MLZero* and *AgentK*, which showed unstable performance and several failed executions on more challenging datasets. Overall, systems capable of leveraging ensemble diversity consistently achieved more stable cross-fold performance and lower variability across benchmarks.

The failure patterns also differed substantially across agents. AutoGluon and MLE-STAR achieved high execution reliability, consistently producing valid outputs across all runs. In contrast, MLZero failed on half of its runs and AgentK on eleven of thirty, while AIDE failed on five. The failure types we observed were exhaustion of the time budget without a submitted prediction file, prediction files that were missing or could not be parsed, schema mismatches between the produced file and the expected test rows, uncaught crashes inside the agent's own code, and hallucinated file paths pointing at inputs that did not exist. We record these qualitatively: our logs identify the terminal state of each run but were not collected in a form that supports a reliable per-type count. Overall, the results suggest that execution robustness and stable search behavior are critical factors for reliable autonomous machine learning systems.

5.2 Runtime, Cost, and Environmental Footprint

Timing was not captured uniformly across the five execution engines, and we report only what was measured. For AutoGluon, scheduler records give wall-clock durations for 31 of the 36 runs of the original twelve-dataset suite. The system exhausted its 7200 s budget on eight of the ten datasets, and converged early on two: *Restaurant Revenue Prediction* in roughly 900 s and *Mercedes-Benz Greener Manufacturing* in roughly 4400 s. Since the budget binds in most cases, this figure characterizes the budget more than the system, but it does establish that the best-ranked system was also the cheapest to run by a wide margin.

For MLE-STAR we can recover only end-to-end elapsed time between successive runs on a fold, which includes scheduler queueing and cannot be separated from it. The median is approximately 5 h and individual gaps exceed the 24 h execution cap, confirming that queueing dominates. We therefore decline to report a mean runtime for MLE-STAR, since the quantity we can measure is not the quantity of interest.

Beyond wall-clock time, the dominant marginal cost of agentic ML systems is token consumption. Every reasoning step, code revision, and recovery attempt consumes backbone LLM tokens, so systems with long refinement loops or high failure rates pay for their instability twice, once in lost runs and once in tokens spent on attempts that produce no valid output. AutoGluon consumes none, and its fixed compute budget reinforces the observation above that the best-ranked system was also the cheapest to run. The asymmetry is also environmental, since LLM inference is energy intensive and token use translates into the footprint of the serving infrastructure [Strubell *et al.*, 2019; Luccioni *et al.*, 2024]. Our execution engine did not capture per-run token counts uniformly. Token accounting should be standard in benchmarks of agentic ML systems, both as a cost measure and as a proxy for environmental impact.

5.3 Summary of Trends

Our experiments reveal several important trends. First, agentic ML systems show strong potential but are highly sensitive to dataset characteristics and execution conditions. Some systems achieve excellent performance on specific datasets while failing on others, which suggests that current gains are not yet robust across problem settings.

Second, execution reliability remains a major challenge. Several systems crashed or returned invalid outputs on selected tasks, highlighting limitations in robustness and workflow handling. These failures are especially common for AgentK and MLZero on larger or more complex datasets.

Third, traditional AutoML approaches such as AutoGluon remain highly competitive against modern agentic systems. Despite lacking explicit reasoning or autonomous planning, AutoGluon often delivers strong predictive performance together with substantially higher reliability.

Overall, these results suggest that current agentic ML systems are promising but still face major challenges in robustness, consistency, and reproducibility. This underscores the need for improved evaluation protocols, stronger execution reliability, and more stable integration between reasoning components and machine learning pipelines.

6 Limitations

Although our benchmarking framework enables a systematic evaluation of agentic machine learning systems, several limitations affect both reliability and practical applicability.

A recurring limitation is instability during long executions, making performance sensitive to runtime budgets and execution conditions. Additionally, many systems are highly sensitive to input data formatting and schema inconsistencies, which can lead to failures or degraded performance.

Another important issue is hallucination. Several systems make incorrect assumptions about file names, dataset structure, or feature semantics even when explicit instructions are provided.

MLZero depends on strong language models and substantial computational resources to achieve competitive performance. In practice, this leads to longer runtimes and higher resource consumption than the other approaches we evaluated, which may limit its use in constrained settings.

Beyond the behavior of individual systems, three limitations constrain what our results can establish.

Scale of the evaluation. Ten datasets is a small suite, and it is the binding constraint on statistical resolution rather than the size of the observed differences: at this scale the Nemenyi critical difference is 1.93 while the full spread of average ranks is 2.05, so only the extremes of the ranking separate. Variance is estimated from three folds per dataset, which is thin. The standard deviation should be read as indicative rather than precise. The corrected score uses it only through a bounded, scale-free term, which limits the influence of a noisy estimate on the ranking, but does not eliminate it.

Comparability across systems. Time budgets were not uniform (Section 3.6), and decoding parameters were each system's defaults rather than a common setting. Both choices are defensible in isolation, and both weaken the claim that we are comparing the five systems on strictly equal terms. For MLE-STAR the comparison is further qualified by our use of an open-source reimplementation whose search backend differs from the original (Appendix A.1).

Backbone mismatch. All systems were evaluated with gpt-oss-120b, a backbone LLM none of them was designed or tuned for. Since prompts and scaffolds are co-designed around specific commercial models, part of the instability and failure behavior we observe plausibly reflects this mismatch rather than the systems themselves. This strengthens the case for automatic configuration: whenever the backbone LLM changes, agentic systems need to be re-adapted, not re-engineered by hand.

Scope. All tasks in this benchmark are tabular, and the task descriptions provided to the agents specify a tabular environment. Several evaluated systems are built for broader modalities, so our findings characterize their behavior in the tabular setting only and do not speak to their performance on image, text, or audio tasks.

Data integrity. Two of our original twelve datasets carried target leakage that survived manual curation, and three folds show signs of inconsistent generation. We have no reason to believe the remaining suite is affected, having audited it, but the episode indicates that curation by inspection is not sufficient and that automated split auditing should be standard practice.

These limitations highlight the need for future work on robustness and budget-aware execution in agentic machine learning systems.

7 Conclusion

In this work, we presented a unified benchmark for evaluating agentic machine learning systems on diverse tabular prediction tasks. Through experiments on a suite of classification and regression datasets, we compared AutoGluon, MLZero, AgentK, AIDE, and an open-source reimplementation of MLE-STAR [WalkingDevFlag, 2025] under a standardized evaluation protocol that measured not only predictive performance but also execution reliability and robustness.

Our results demonstrate that ensemble-based systems currently provide the most stable and reliable performance across heterogeneous datasets. AutoGluon consistently achieved strong predictive performance together with the highest execution reliability, while AIDE showed competitive performance through efficient iterative refinement. MLZero occasionally achieved excellent task-specific results, but suffered from instability and frequent execution failures. AgentK demonstrated weaker generalization and robustness across multiple tasks, whereas MLE-STAR achieved moderate but reliable performance across the benchmark.

The experiments further reveal that current agentic ML systems remain highly sensitive to dataset characteristics, execution conditions, and search stability. Our findings suggest that reliable execution, stable search behavior, and ensemble diversity are critical for building practical autonomous machine learning systems, and we hope this benchmark provides a useful foundation for future work on them.

Our results also point to automatic configuration of agentic ML systems as a central open problem, since prompts and component choices must be re-optimized whenever the backbone LLM changes. Automated prompt optimization and its multi-objective extension to inference cost [Zehle *et al.*, 2025; Büssing *et al.*, 2026], and approaches that optimize entire LLM programs [Opsahl-Ong *et al.*, 2024], are first steps in this direction. Knowing which component to update is itself addressed by recent work on contrastive credit attribution [Zehle, 2026]. A second extension is multi-modality: several evaluated systems target image, text, and audio tasks, while our benchmark specifies a tabular environment, and extending it would test whether these patterns hold.

Acknowledgments

Kartik Nagaraj Nayak is supported by the Konrad Zuse School of Excellence in Learning and Intelligent Systems (ELIZA) through the DAAD programme Konrad Zuse Schools of Excellence in Artificial Intelligence, sponsored by the Federal Ministry of Education and Research.

Tom Zehle, Omar Swelam and Alexander Pfefferle acknowledge funding by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Commission. Neither the European Union nor the European Commission can be held responsible for them. This work was supported by the European Union's Horizon Europe research and innovation programme under grant agreement No 101214398 (ELLIOT). Frank Hutter acknowledges funding by the Hector Foundation.





References

[Bergstra *et al.*, 2011] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 24. Curran Associates, Inc., 2011.

[Büssing *et al.*, 2026] Jan Büssing, Moritz Schlager, Timo Heiß, Tom Zehle, and Matthias Feurer. MO-CAPO: Multi-objective cost-aware prompt optimization. arXiv preprint arXiv:2605.18869, 2026.

[Chan *et al.*, 2025] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. MLE-bench: Evaluating machine learning agents on machine learning engineering. arXiv preprint arXiv:2410.07095, 2025.

[Erickson *et al.*, 2020] Nick Erickson, Jonas Mueller, Alexander Shirkov, Hang Zhang, Pedro Larroy, Mu Li, and Alexander Smola. AutoGluon-Tabular: Robust and accurate AutoML for structured data. *arXiv preprint arXiv:2003.06505*, 2020.

[Fang *et al.*, 2025] Haoyang Fang, Boran Han, Nick Erickson, Xiyuan Zhang, Su Zhou, Anirudh Dagar, Jiani Zhang, Ali Caner Turkmen, Cuixiong Hu, Huzefa Rangwala, Ying Nian Wu, Bernie Wang, and George Karypis. MLZero: A multi-agent system for end-to-end machine learning automation. arXiv preprint arXiv:2505.13941, 2025.

[Feurer *et al.*, 2020] Matthias Feurer, Katharina Eggensperger, Stefan Falkner, Marius Lindauer, and Frank Hutter. Auto-Sklearn 2.0: Hands-free AutoML via meta-learning. *arXiv preprint arXiv:2007.04074*, 2020.

[Gijsbers *et al.*, 2024] Pieter Gijsbers, Marcos L. P. Bueno, Stefan Coors, Erin LeDell, Sébastien Poirier, Janek Thomas, Bernd Bischl, and Joaquin Vanschoren. AMLB: an AutoML benchmark. *Journal of Machine Learning Research*, 25(101):1–65, 2024.

[Grosnit *et al.*, 2025] Antoine Grosnit, Alexandre Maraval, Refinath S N, Zichao Zhao, James Doran, Giuseppe Paolo, Albert Thomas, Jonas Gonzalez, Abhineet Kumar, Khyati Khandelwal, Abdelhakim Benechehab, Hamza Cherkaoui, Youssef Attia El-Hili, Kun Shao, Jianye Hao, Jun Yao, Balázs Kégl, Haitham Bou-Ammar, and Jun Wang. Kolb-based experiential learning for generalist agents with human-level Kaggle data science performance. arXiv preprint arXiv:2411.03562, 2025.

[Huang *et al.*, 2024] Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. arXiv preprint arXiv:2310.03302, 2024.

[Jiang *et al.*, 2025] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.

[Luccioni *et al.*, 2024] Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power hungry processing: Watts driving the cost of AI deployment? In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024.

[Nam *et al.*, 2025] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Ö. Arık, and Tomas Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement. arXiv preprint arXiv:2506.15692, 2025.

[Nathani *et al.*, 2025] Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. MLGym: A new framework and benchmark for advancing AI research agents. arXiv preprint arXiv:2502.14499, 2025.

[Olson and Moore, 2016] Randal S. Olson and Jason H. Moore. TPOT: A tree-based pipeline optimization tool for automating machine learning. In Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren, editors, *Proceedings of the Workshop on Automatic Machine Learning*, volume 64 of *Proceedings of Machine Learning Research*, pages 66–74, New York, New York, USA, 24 Jun 2016. PMLR.

[OpenAI, 2025] OpenAI. gpt-oss-120b. `https://huggingface.co/openai/gpt-oss-120b`, 2025. Accessed: 2026-08-11.

[Opsahl-Ong *et al.*, 2024] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and

demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, 2024.

[Shinn *et al.*, 2023] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. arXiv preprint arXiv:2303.11366, 2023.

[Strubell *et al.*, 2019] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019.

[Vanschoren *et al.*, 2014] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: networked science in machine learning. *ACM SIGKDD Explorations Newsletter*, 15(2):49–60, June 2014.

[WalkingDevFlag, 2025] WalkingDevFlag. MLE-STAR-Open. `https://github.com/WalkingDevFlag/MLE-STAR-Open`, 2025. Accessed: 2026-05-21.

[Wang *et al.*, 2019] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. arXiv preprint arXiv:1804.07461, 2019.

[Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2023.

[Zehle *et al.*, 2025] Tom Zehle, Moritz Schlager, Timo Heiß, and Matthias Feurer. CAPO: Cost-aware prompt optimization. In *4th International Conference on Automated Machine Learning*, 2025.

[Zehle, 2026] Tom Zehle. CANTANTE: Optimizing agentic systems via contrastive credit attribution. arXiv preprint arXiv:2605.13295, 2026.

[Zoph and Le, 2017] Barret Zoph and Quoc V. Le. Neural architecture search with reinforcement learning. arXiv preprint arXiv:1611.01578, 2017.

Appendix

A MLE-STAR

MLE-STAR is a multi-agent framework designed to solve complex machine learning tasks across diverse data modalities, including tabular, image, text, and audio data. Its operation begins with a "Search-as-a-Tool" initialization phase, where the agent queries the web to retrieve task-specific models and code examples to generate a high-quality initial solution, thereby avoiding the suboptimal model suggestions often made by LLMs relying solely on internal knowledge. Once an initial script is produced, MLE-STAR employs a dual-loop optimization strategy: an outer loop performs ablation studies to identify the specific code blocks or ML components (such as imputation methods or feature encoding) that most significantly impact performance, while an inner loop iteratively refines only those targeted blocks.

A key special feature of MLE-STAR is its ability to extract and transform localized code segments using specialized agents, which are $A_{extractor}$ for identifying target blocks and A_{coder} for implementing refinement plans, without rewriting the entire script. Furthermore, the system incorporates a novel ensemble mechanism where a planning agent ($A_{planner}$) proposes and evaluates parallel ensemble strategies to integrate multiple solutions into a final, optimized model. To ensure methodological integrity, the framework also includes dedicated modules to mitigate undesirable behaviors like data leakage, such as the improper use of test sample statistics during missing value imputation.

A.1 Reimplementation used in this study

We evaluate the open-source reimplementation of MLE-STAR [WalkingDevFlag, 2025] rather than the original release, which depends on Google's Agent Development Kit and on Google Search. The reimplementation replaces the Agent Development Kit and the `google.genai` client with minimal local shims and routes all model calls through a generic OpenAI-compatible client, which is what allows every system in this study to share one backbone LLM. It also substitutes DuckDuckGo for Google Search.

The search substitution deserves a caveat. MLE-STAR's initialization phase retrieves task-specific models and code examples from the web, so a weaker or differently ranked search backend may weaken that initialization relative to the original system. We cannot quantify the effect without rerunning the original release, which the Google dependencies prevent us from doing under our network restrictions. Results reported here should therefore be read as those of the reimplementation, not of MLE-STAR as published.

B Leakage Audit Forensics

The audit checks five conditions on every fold: presence of the target column itself in the test file, exact reconstruction of the target from any single feature, exact reconstruction from any additive pair of features, schema mismatch between the train and test files of a fold, and row overlap between the train and test splits. It is released with the benchmark as `src/audit_splits.py` so that the check can be rerun and extended.

In *Bike Sharing Demand*, the features `casual` and `registered` are retained in the test file on all three folds and sum exactly to the target `count`. Their sum reproduces the held-out target on all 10,886 test rows, for a coefficient of determination of 1.000000. A third, harmless inconsistency of the same kind appears in the same dataset, whose first fold names the target column differently from the other two.

In *Bioresponse*, the test file of the first fold retains the `Target` column itself, matching the ground truth exactly. That fold also differs from the other two in its schema, carrying 1778 columns rather than 1777 and a lower-case identifier column, indicating that it was produced by a different code path.

Three of our folds therefore show evidence of inconsistent generation, and the leaked features in *Bike Sharing Demand* are a documented property of the source dataset that survived our curation. We draw a methodological conclusion from this rather than treating it as an isolated defect: curation by inspection is not sufficient, and automated split auditing should be standard practice.

C Dataset Overview

Dataset	Task type	Metric	Source (Kaggle competition)
Santander Customer Satisfaction	Binary	ROC-AUC	santander-customer-satisfaction
Forest Cover Type Prediction	Multiclass	Multiclass ROC-AUC	forest-cover-type-prediction
Give Me Some Credit	Binary	ROC-AUC	GiveMeSomeCredit
Otto Group Product Classification	Multiclass	Multiclass ROC-AUC	otto-group-product-classification-challenge
Porto Seguro Safe Driver Prediction	Binary	ROC-AUC	porto-seguro-safe-driver-prediction
Predict Who Is More Influential	Binary	ROC-AUC	predict-who-is-more-influential-in-a-social-network
Home Data for ML Course	Regression	R^2 and RMSE	home-data-for-ml-course
Mercedes-Benz Greener Manufacturing	Regression	R^2 and RMSE	mercedes-benz-greener-manufacturing
Restaurant Revenue Prediction	Regression	R^2 and RMSE	restaurant-revenue-prediction
Sberbank Russian Housing Market	Regression	R^2 and RMSE	sberbank-russian-housing-market

Table 3: The 10 datasets forming the evaluation suite after the leakage audit, with task type, evaluation metric, and source competition. Each dataset is split into three fixed folds, and every fold into a train and a test partition.

The full dataset descriptions, exactly as passed to the agents, are available with the released code at `github.com/KARTIK-N-NAYAK/Benchmarking-Agentic-Data-Scientists`.

D Results

Comparison	Raw p	Holm p	BH p
AutoGluon vs MLZero	0.018	0.176	0.137
AutoGluon vs MLE-STAR	0.039	0.352	0.137
AutoGluon vs AgentK	0.041	0.352	0.137
AIDE vs MLZero	0.086	0.602	0.215
AgentK vs AIDE	0.277	1.000	0.421
MLZero vs MLE-STAR	0.289	1.000	0.421
AutoGluon vs AIDE	0.295	1.000	0.421
AgentK vs MLE-STAR	0.551	1.000	0.675
AIDE vs MLE-STAR	0.607	1.000	0.675
AgentK vs MLZero	0.779	1.000	0.779

Table 4: Pairwise Wilcoxon signed-rank test results on per-dataset method ranks, ordered by raw p-value. Significance was assessed at $\alpha = 0.05$ after correction for multiple comparisons. No comparison is significant under either correction.